

Programación de Objetos Distribuidos Trabajo

Práctico 2: 'Reclamos Urbanos'



Integrantes del grupo:

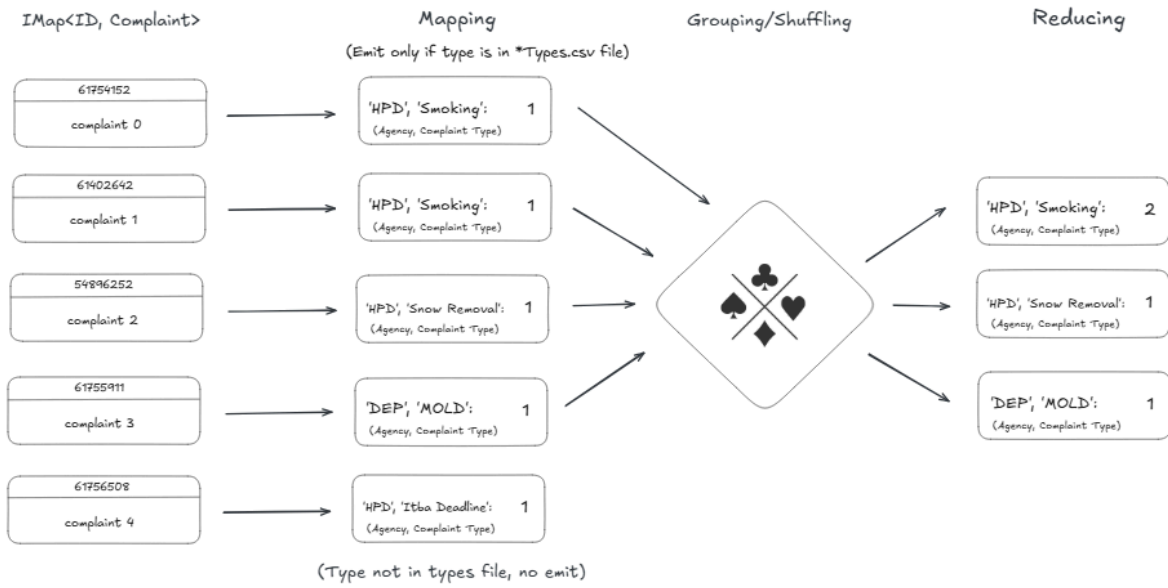
Bernardo Zapico - 62318
Tomás Guillermo Sheffer - 63393
Alejo Padula Morillo - 63571



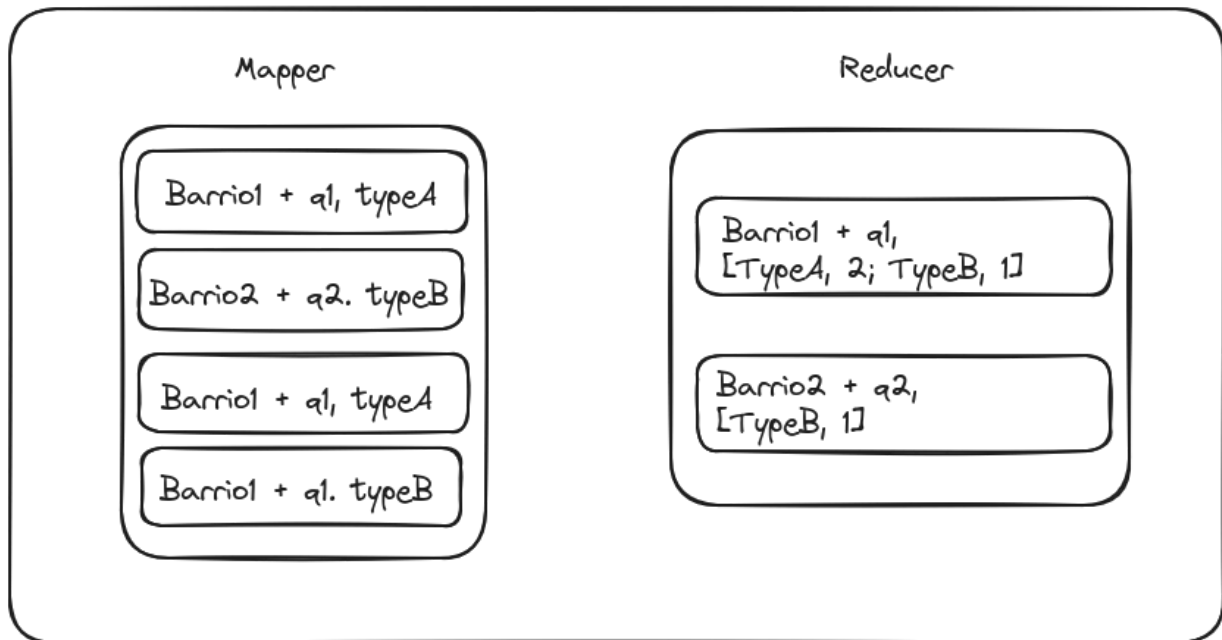
Diseño

Al comienzo del trabajo realizamos las configuraciones del Server y naturalmente, empezamos con la implementación de la Query 1. Durante el desarrollo de esta primero encontramos necesario modelar los reclamos (Complaints) para facilitarnos luego el Parseo de los archivos 'csv' a un modelo de java, luego hicimos un primer parser que realizaba una lectura completa del archivo y se le pasaba una ICollection para insertar sobre las mismas los Complaints, ambas cosas fueron 'obligatoriamente' cambiadas/arregladas. Terminamos decidiendo pasarle al parser un Consumer<Complaint> para manejar la inserción de cada Complaint de la forma que deseamos. Básicamente, desde las queries declararemos la colección que deseamos y luego en la expresión lambda del Consumer<Complaint> lo insertamos de la forma que nos convenga (eligiendo los pares clave/valor más útil para la query). Luego continuamos con la implementación de las respectivas Queries:

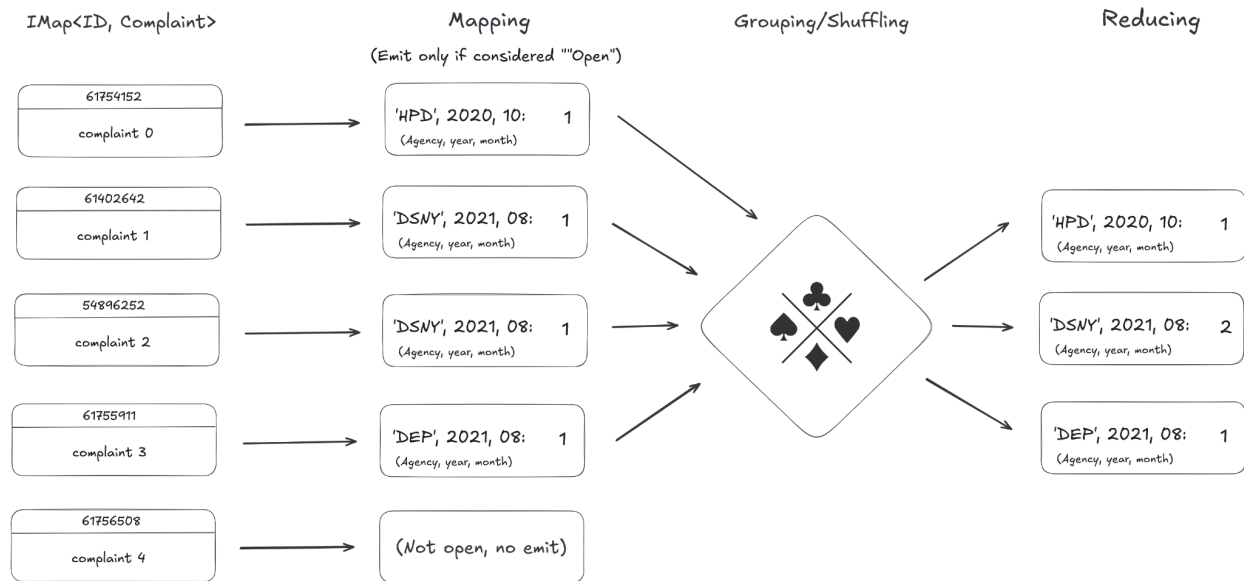
Para la Query 1 como lo que se pide es un total de reclamos por agencia implementamos un Mapper y Reducer similar al que vimos en clase con el 'Word Count'. El Mapper recibe los Complaints y los convierte en un objeto "Pair" de agencia y tipo de reclamo (Key) y un Integer de valor fijo 1 (Value). Ya desde el Reducer solo nos dedicamos a contar la cantidad de Keys iguales. Como todavía tenemos el trabajo post-procesado de ordenar estos resultados un Collator fue la primer y mejor solución vista (esta se encarga únicamente del ordenado).



Para el Query 2, decidimos primero pasarle al mapper un MultiMap de las coordenadas en los cuadrantes calculados + el vecindario, en una clase `CoordinateNeighborhood` (key) y sus complaints (value). Esto facilita el trabajo del mapper ya que solo tiene que emitir el `CoordinateNeighborhood` de clave y el tipo de problema de value. El Reducer tiene el trabajo de, para cada `CoordinateNeighborhood`, agrupar en su value los tipos de problemas y sumar sus apariciones, de esa forma se puede buscar fácilmente cuál es el tipo que más aparece para ese `CoordinateNeighborhood`. Luego el collator recorre el mapa resultante y se queda con el tipo más común, finalmente también ordena la salida.



Para la Query 3 optamos por realizar el filtrado de reclamos abiertos directamente en el Mapper para emitir ya desde esta etapa solo lo necesario para producir la respuesta apropiada. Para realizar la agrupación al inicio pensamos en generar un string formado por la concatenación de agencia, año y mes, pero al final optamos generar la clase 'MonthCount' para trabajar estos datos más fácilmente. Con este Mapper simplificamos mucho del lado del Reducer ya que su único trabajo es recibir estas agrupaciones, iniciar un contador e incrementarlo por cada entrada (Simil Q1). Para realizar los cálculos de promedios y ordenar la respuesta decidimos usar un Collator el cual debe generar una lista con todas las agencias y agrupar por las mismas una lista ordenada con los meses y total de reclamos en cada una. Este Collator tuvo en complejidad el cálculo del promedio de reclamos pues tenemos que tomar en consideración que existen 'huecos' en las listas dado por los meses en los que una agencia no recibió reclamos (pero si nos interesa promediarlos y también mostrar el promedio que generan).



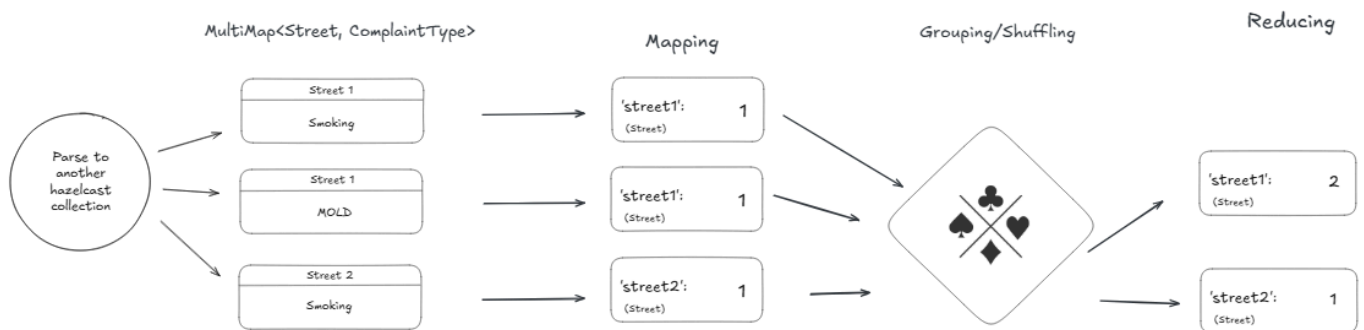
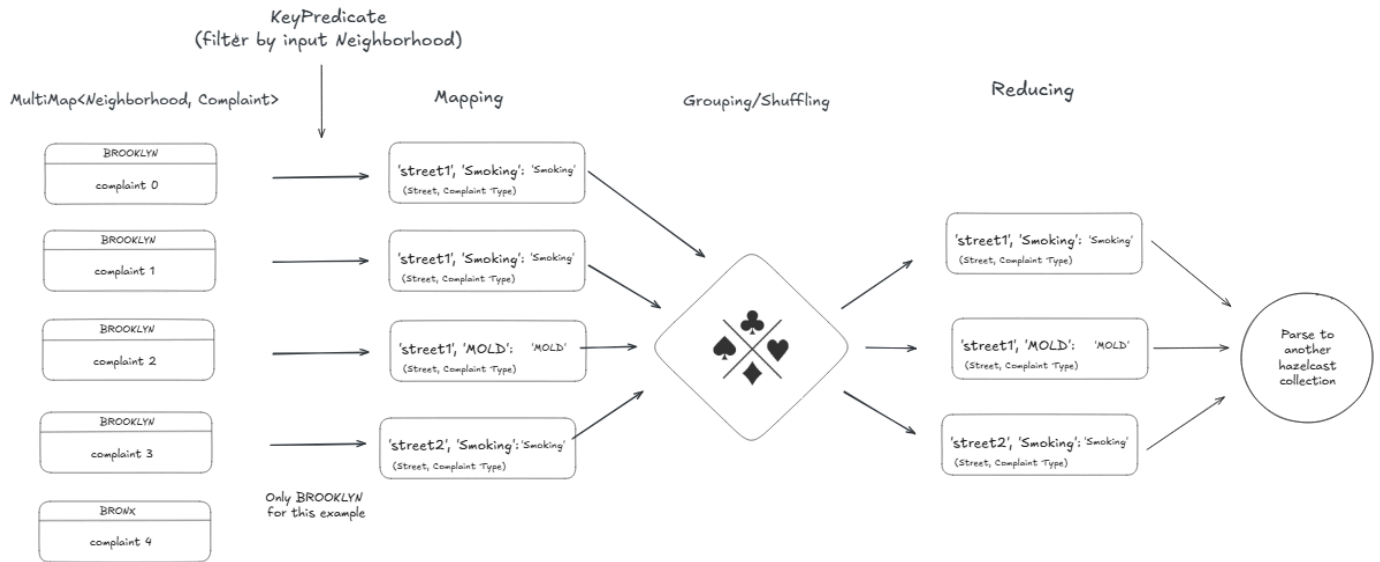
Para la query 4 primero utilizamos un MultiMap<Neighborhood, Complaint> donde se guardará la información parseada del 'csv', la ventaja de este MultiMap es que podemos usar el KeyPredicate para filtrar por barrio (con el argumento que se ingresa en la Query).

Una vez filtradas las entradas a las de ese barrio, empezamos a pensar la estructura de MapReduce que íbamos a utilizar. Para esta Query, pensamos dividir el trabajo en dos Jobs distintos, uno que se encargue de eliminar las entradas que tengan la misma calle y tipo de reclamo y otro que se encargue de contar los tipos de reclamos totales para esa calle.

Para el primer Job se decidió emitir de clave <Calle + Tipo de reclamo, Tipo de Reclamo>, aunque el value que se eligiera no es tan importante en este caso. Lo importante al reducir es quedarse con una única entrada de Calle + Tipo de reclamo para que en el siguiente Job pueda contar los tipos de reclamos distintos. Una vez terminado el primer Job, se parsea el output a otra colección de hazelcast, en este caso un MultiMap, con la calle como Key y de Value el tipo de reclamo.

Para el segundo Job se emitió en el Mapper la calle (key) y un 1 (value), con el objetivo de que en el Reducer se cuente la cantidad de tipos de reclamos distintos,

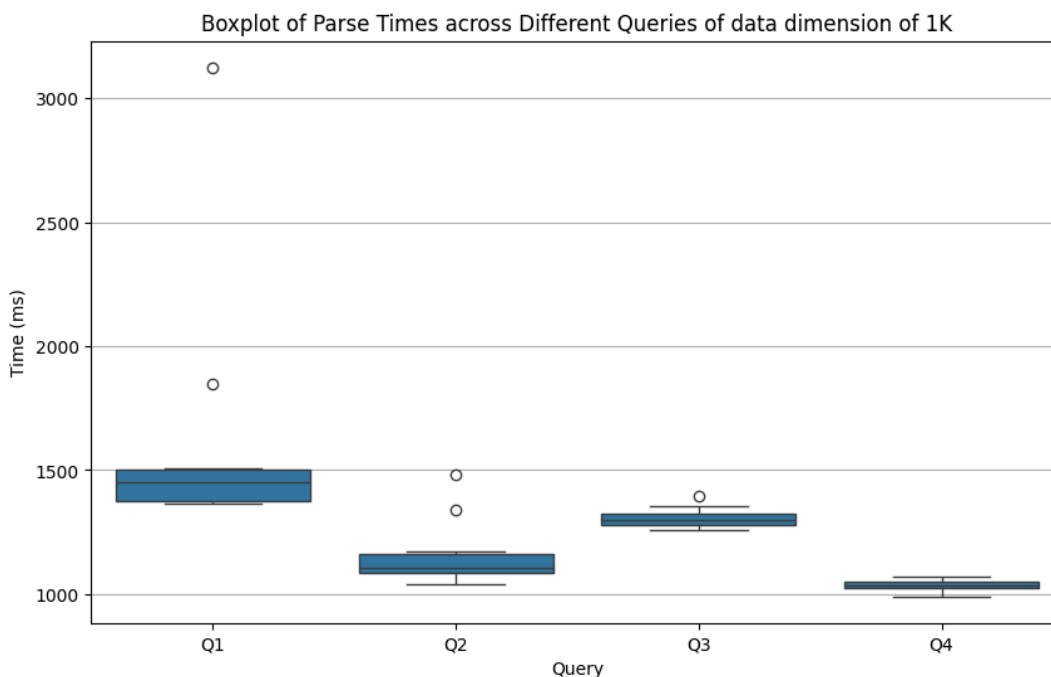
algo similar a lo que se hizo en el Query 1. Finalmente se ordena el resultado utilizando un Collator.



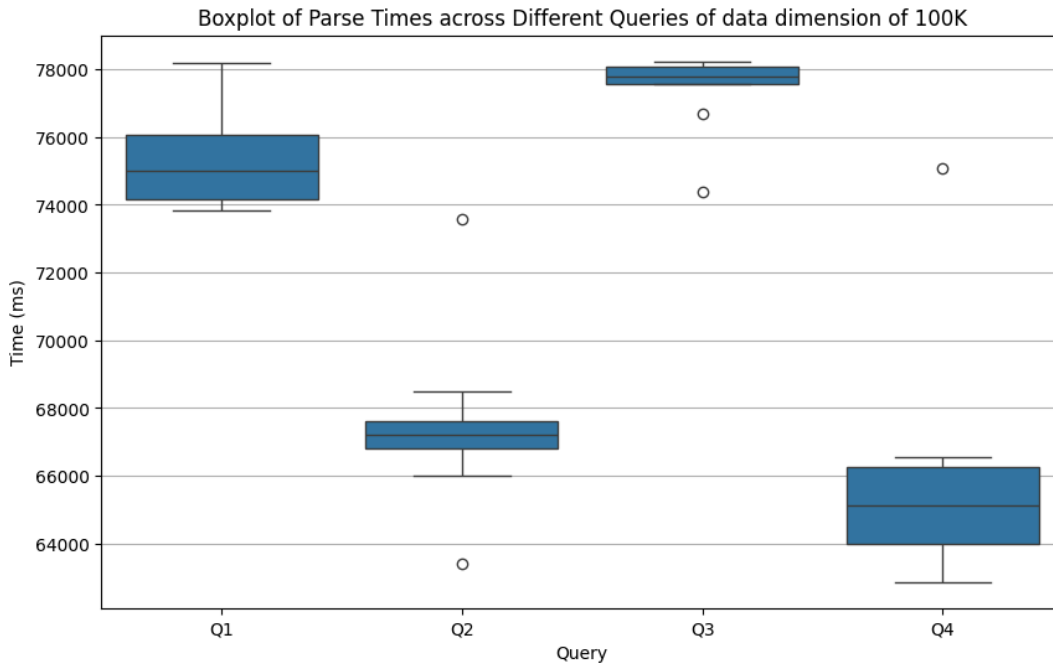
Análisis de tiempos

Para los análisis de tiempos se tomaron ciertos datos como fijos para concentrar el análisis en cómo afecta un parámetro a la vez. La cantidad de datos se mencionará en los títulos de los gráficos. Se tomó:

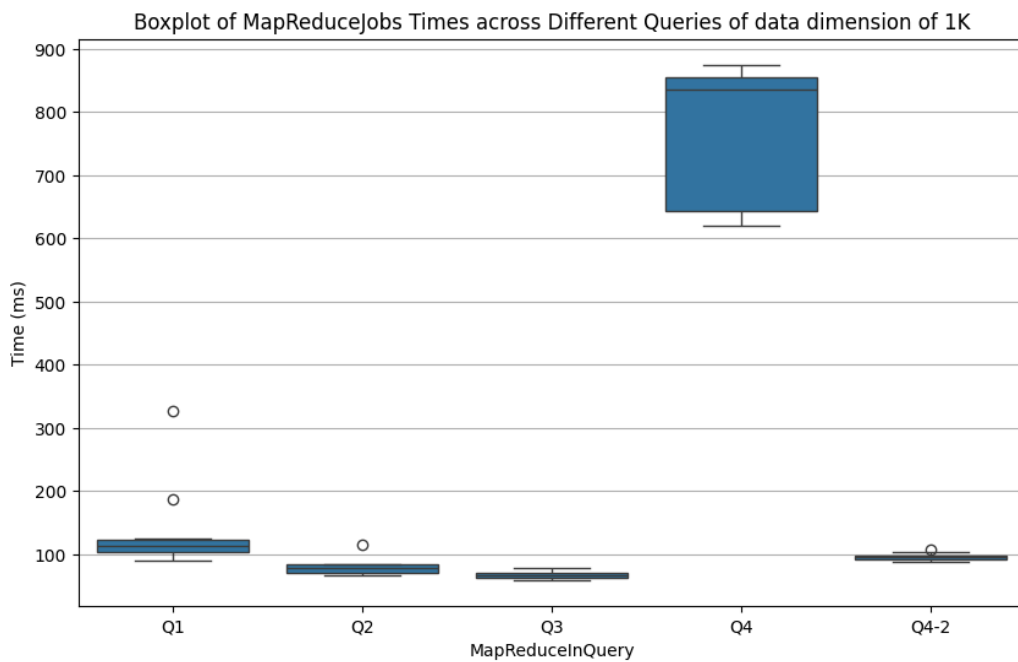
- La ciudad de Nueva York del dataset de la cátedra
- Para la query 2 se optó por un $q = 0.1$
- Para la query 3 se optó por un $w = 3$
- Para la query 4 se optó por un neighbourhood = BRONX



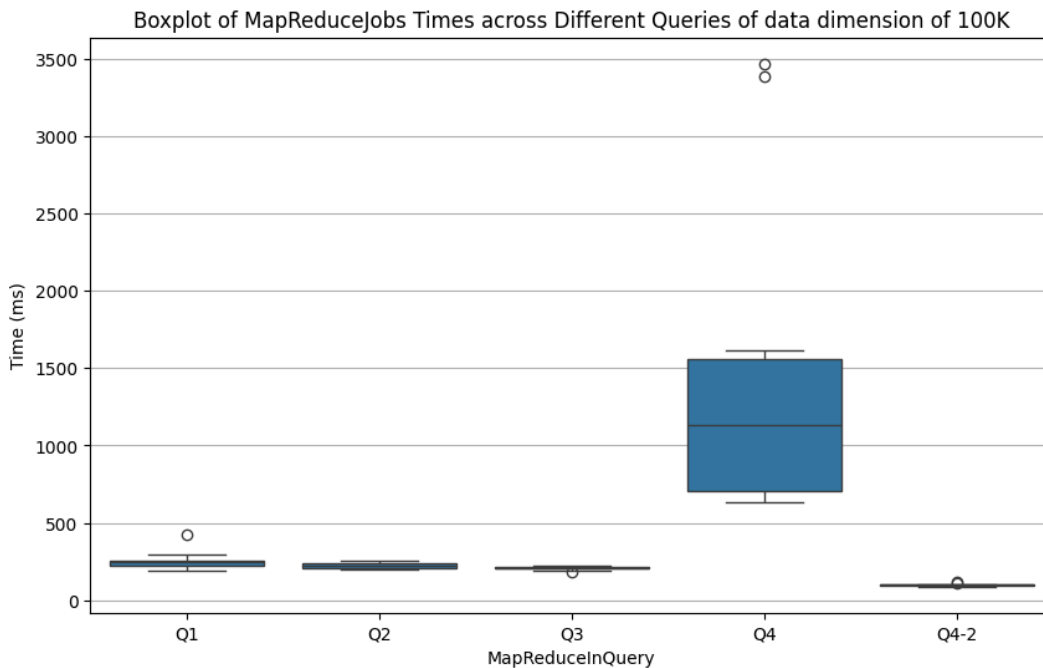
Como se puede observar, para un volumen de mil datos hay una ligera variación en lo que se tarda en parsear las entradas a procesarse pero se mantiene bastante homogéneo.



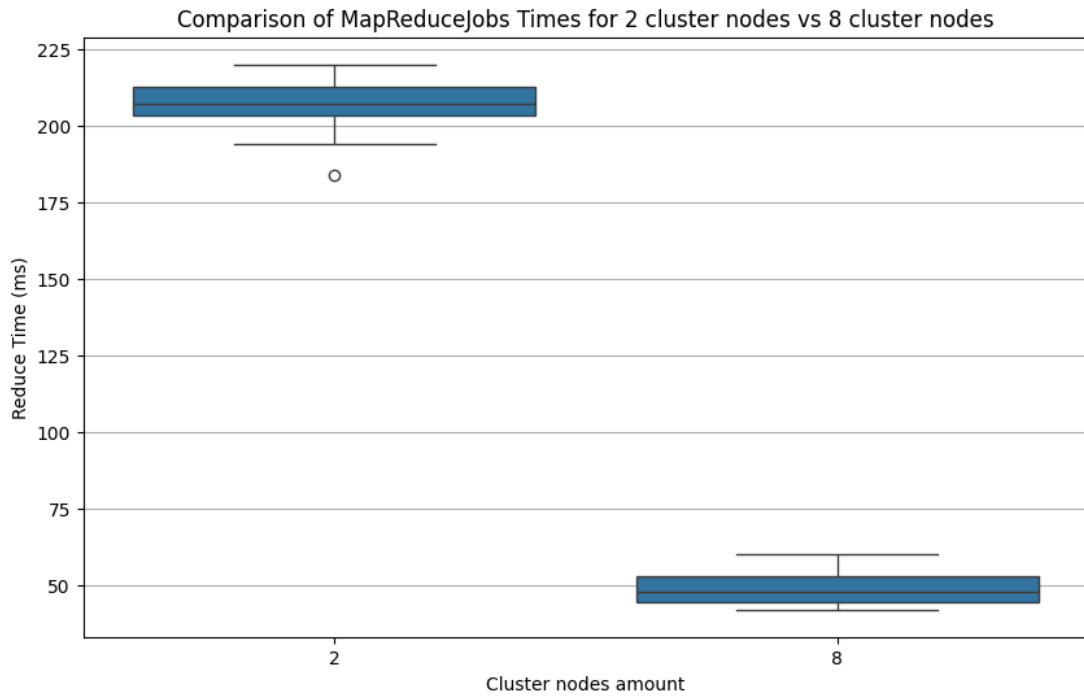
Al manipular datos con un volumen dos órdenes de magnitud más grande, se puede apreciar que el tiempo de parseo se extiende casi en la misma proporción.



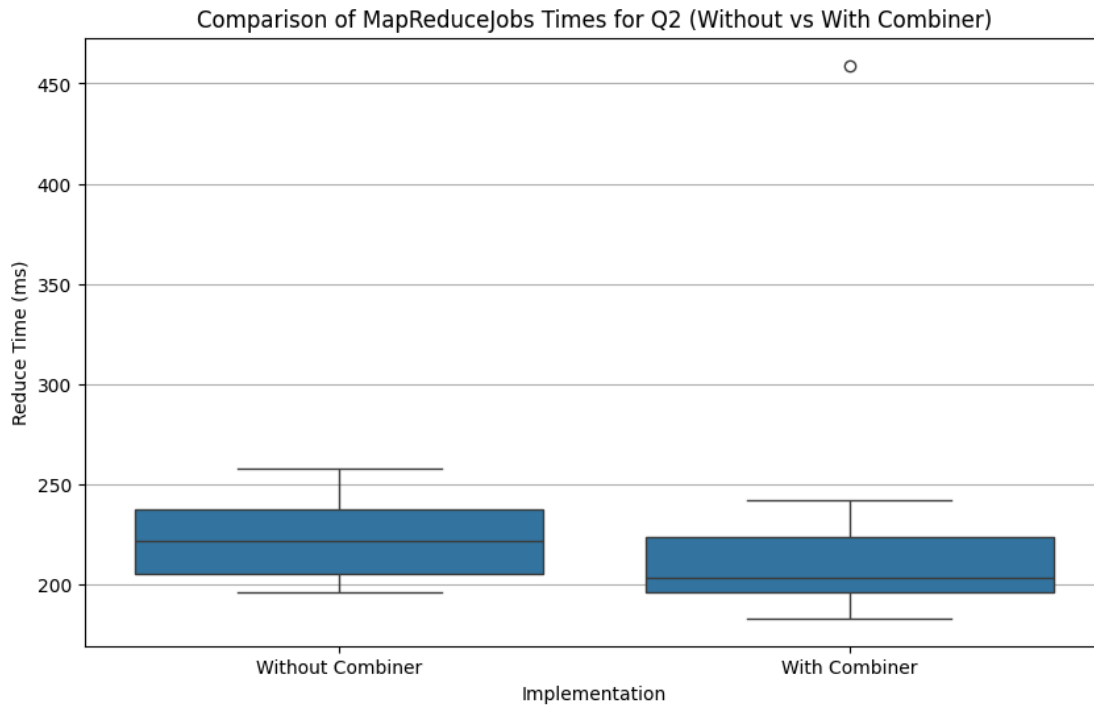
Se puede observar que todas las operaciones de MapReduce tienen un runtime parecido con la excepción de la primera parte del Query 4 que tiene un tiempo notablemente extendido en comparación al resto.



Notablemente, a pesar de que el volumen de datos es 100 veces más grande, el tiempo de ejecución del proceso de MapReduce no se incrementa de la misma forma. Es decir, lo que más se ve afectado por un mayor volumen de datos es el parseo y no tanto las operaciones MapReduce.



Para esta comparación se tomó la Query 3. Hay una diferencia inmediatamente visible entre lo que se demora el MapReduce cuando se usa 2 nodos en el cluster cuando usa 8 (realizado con 100K registros).



Para esta comparación se utilizó la Query 2. Hay una diferencia muy ligera entre el uso y el no-uso del combiner. Aunque el combiner tiene una ligera ventaja en esta prueba, el margen es demasiado corto para sacar conclusiones. (1K registros)

Puntos de mejora y expansión

Una mejora puede ser como no se usa toda información levantada desde el Parser, que cada Query solo recopile la información que necesita y descartar el resto. O sea, realizar un Parseo personalizado por Query, y de esta manera podremos agilizar los tiempos de carga a costa de mayor complejidad y menor reutilización de código.

También pensamos que se podría haber experimentado más con el uso de combiners para ver si en realidad puede mejorar el tiempo de ejecución de las tareas de map-reduce.

Otra mejora que no terminamos implementando (Query 3) es prefiltrar con el uso de un Predicate el estado del reclamo para que aquellas consideradas no abiertas (Open) directamente no pasen por el MapReduce.

Una notable mejora sería buscar modularizar un poco más el código de los clientes, si bien se hace bastante con la conexión al hazelcast y validación de argumentos globales, podría lograrse algo más genérico.

Por último como expansión pensamos que resultaría útil, por lo menos para una Query que puede demorar mucho tiempo, obtener alguna especie de métrica de avance sobre la misma.